作业 HW2：分治算法

课程：算法分析与设计
作业编号：HW2
对应模块：分治
截止时间：2026-09-19
提交方式：发送压缩文件夹到965877896@qq.com，邮件主题：作业 HW2分治算法学号姓名，附件压缩文件名：HW2 _学号 _姓名。

一、作业目的

1. 掌握**分治算法**的“分解（Divide）–解决（Conquer）–合并（Combine）”三步骤核心思想。
2. 理解并实现**平面最近点对问题**的 O(n log n) 分治算法。
3. 体验**五种不同提示策略**在分治算法学习中的效果差异。

二、通用规则

2.1 编程语言与 AI 工具

- **编程语言**：任选（Python / Java / C++ 等）。
- **AI 工具**：任选（ChatGPT / Claude / DeepSeek / Copilot / 文心一言 等）。不同策略可以使用不同工具，但须在 `metadata.json` 中分别注明具体工具和模型版本（如 `ChatGPT-4o`、`DeepSeek-R1`）。

2.2 五个策略的通用执行规则

1. **每个策略必须开启一次全新的对话**（清空上下文 / 点击“New Chat”）。
2. **每个策略下只能输入一次提示词**，不得追问或补充信息。
3. **不得修改提示词中的任何文字**，仅将 `[你的语言]` 替换为实际编程语言名（如 `Python`、`Java`、`C++`）。
4. 如果 AI 的回答分多段输出（如流式输出），全部合并记录为一次回答。
5. **策略 S4 和 S5 内部的三步是一次 AI 调用中完成的**，不是三次独立的 AI 调用。你输入一次提示词，AI 按三步结构输出完整回答。

三、算法题目

平面最近点对问题（Closest Pair of Points）

问题描述：

给定平面上 n 个点（n ≥ 2），找出欧几里得距离最近的一对点。

定义：点 $p_1(x_1, y_1)$ 和 $p_2(x_2, y_2)$ 之间的欧几里得距离为：

$d(p_1, p_2) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$

输入格式：

```
n
x₁ y₁
x₂ y₂
...
xₙ yₙ
```

- 第一行为整数 n（$2 \le n \le 10^5$），表示点的数量。
- 接下来 n 行，每行两个**整数**，表示一个点的 x 和 y 坐标。
- 所有点的坐标互不相同。

输出格式：

```
最近距离: d
点1: (x₁, y₁)
点2: (x₂, y₂)
```

- d 为最近点对的欧几里得距离，**保留 6 位小数**。
- 输出的一对点中，**先输出 x 坐标较小的点**；若 x 坐标相同，则先输出 y 坐标较小的点。
- **如果存在多对距离相同的最近点对，输出任意一对即可**，但须满足上述点内排序规则。

示例：

输入：

```
6
0 0
1 1
2 2
3 3
4 4
5 5
```

输出：

```
最近距离: 1.414214
点1: (0, 0)
点2: (1, 1)
```

你的任务：

实现分治算法求解平面最近点对问题，要求**时间复杂度为 O(n log n)**。

算法核心步骤：

1. **预处理**：将所有点按 x 坐标升序排序。
2. **分解（Divide）**：如果当前点集点数 ≤ 3，直接使用蛮力法计算最近点对。否则，按 x 坐标中位数将点集划分为左右两个子集 PL 和 PR。
3. **解决（Conquer）**：递归地在 PL 和 PR 中分别求解最近点对，得到距离 dL 和 dR，令 d = min(dL, dR)。
4. **合并（Combine）**：
   - 构造**带状区域（strip）**：收集所有与分割线（中位线）的 x 方向距离小于 d 的点。
   - 将带状区域内的点按 y 坐标升序排序。
   - 对带状区域中的每个点，**只需检查其后的最多 7 个点**（具体实现中检查索引差 ≤ 7 或索引差 < 8 均可，两种写法均正确），计算距离并更新 d。
5. 返回最终的最短距离 d 及对应的点对。

> **关键证明**：在宽度为 2d 的带状区域内，当点按 y 坐标排序后，对于任意一个点，只需要检查**最多 7 个后续点**即可保证找到跨左右子集的最近点对。这是分治算法能够达到 O(n log n) 时间复杂度的关键。

四、五种提示策略

策略 S1：

你须输入的提示词（将 `[你的语言]` 替换为你的编程语言）：

> 请用 [你的语言] 实现平面最近点对问题的分治算法。给定平面上 n 个点，找出欧几里得距离最近的一对点，输出最近距离和对应的两个点。要求时间复杂度 O(n log n)。
>
> 你的回答中，**除了代码本身之外，不得包含任何解释、推理或说明文字。代码内部也不得包含任何注释（即不得出现 `//` 或 `#` 等注释符号）**。只返回完整的代码。

策略 S2：

你须输入的提示词（将 `[你的语言]` 替换为你的编程语言）：

> 请用 [你的语言] 实现平面最近点对问题的分治算法。给定平面上 n 个点，找出欧几里得距离最近的一对点，输出最近距离和对应的两个点。要求时间复杂度 O(n log n)。
>
> 在给出完整代码之前，**请分步展示你的推理过程**，包括：
>
> 1. 分治算法的"分解（Divide）–解决（Conquer）–合并（Combine）"三步骤分别是什么？
> 2. 为什么在合并步骤中只需要检查带状区域（strip）内的点？
> 3. 为什么在带状区域内每个点最多只需要检查 7 个后续点？这个结论的数学依据是什么？
> 4. 算法的整体时间复杂度为什么是 O(n log n)？
>
> 完成上述推理后，再给出完整代码。

策略 S3：

你须输入的提示词（将 `[你的语言]` 替换为你的编程语言）：

> 请用 [你的语言] 实现平面最近点对问题的分治算法。给定平面上 n 个点，找出欧几里得距离最近的一对点，输出最近距离和对应的两个点。要求时间复杂度 O(n log n)。
>
> 你的回答**必须严格按照以下五部分结构组织**：
>
> **第一部分：算法思路** — 用自然语言描述分治算法的核心思想，包括分解、解决、合并三个步骤。
>
> **第二部分：伪代码** — 用清晰的伪代码表示算法的完整逻辑流程。
>
> **第三部分：完整代码** — 用 [你的语言] 给出可运行的完整代码，包含必要的注释。
>
> **第四部分：复杂度分析** — 推导算法的时间复杂度和空间复杂度，写出递推式并求解。
>
> **第五部分：正确性证明的关键** — 解释为什么合并步骤中只需要检查带状区域内按 y 排序后的最多 7 个点。

策略 S4：

你须输入的提示词（将 `[你的语言]` 替换为你的编程语言）：

> 请用 [你的语言] 实现平面最近点对问题的分治算法。给定平面上 n 个点，找出欧几里得距离最近的一对点，输出最近距离和对应的两个点。要求时间复杂度 O(n log n)。
>
> 请**按以下三步顺序完成**：

> **第一步：初始代码** — 给出一个可运行的完整代码。代码外部不得附加任何解释或说明文字，代码内部也不得包含任何注释。
>
> **第二步：批判性分析** — 对你刚给出的代码进行批判性分析：
>
> 1. 该实现的主要局限性是什么？（如：是否真正达到了 O(n log n)？常数因子是否过大？边界条件处理是否完善？）
> 2. 在合并步骤中，你是如何实现“检查最多 7 个点”的？这个实现是否严格遵循了理论证明？
> 3. 代码在什么极端情况下可能出错或性能下降？（如：所有点 x 坐标相同、所有点 y 坐标相同、n 为 2 或 3 等）
>
> **第三步：改进版代码** — 基于以上批判，给出一个改进后的版本，并逐条说明每处改进的原因。

策略 S5：

你须输入的提示词（将 `[你的语言]` 替换为你的编程语言）：

> 请用 [你的语言] 实现平面最近点对问题的分治算法。给定平面上 n 个点，找出欧几里得距离最近的一对点，输出最近距离和对应的两个点。要求时间复杂度 O(n log n)。
>
> 请**按以下三步顺序完成**：
>
> **第一步：标准代码** — 给出一个标准、高效的实现版本，包含必要的注释。
>
> **第二步：教学反思（以教师视角）** —
>
> 1. 这段代码中最可能让初学者困惑的三个地方分别是什么？为什么？
> 2. 针对每个困惑点，你会如何向学生讲解才能帮助他们理解？
> 3. 初学者在实现分治最近点对时最常见的错误有哪些？如何避免？
>
> **第三步：教学版代码** — 基于以上反思，重新输出一个面向初学者的教学版代码，要求：
>
> - 每行关键代码都有详细的注释，解释“这段代码在做什么”和“为什么这样做”
> - 在代码顶部添加“算法思想概览”的注释块，清晰标出“分解”“解决”“合并”三个阶段
> - 在带状区域处理和“7 个点”检查的关键代码处添加“⚠ 注意”标签
> - 代码必须是可运行的

五、提交物清单

```
HW2_学号_姓名.zip
├── metadata.json
├── strategy_S1/
|   ├── dialogue.txt
```

```
|   └── final_code.xxx
├── strategy_S2/
|   ├── dialogue.txt
|   └── final_code.xxx
├── strategy_S3/
|   ├── dialogue.txt
|   └── final_code.xxx
├── strategy_S4/
|   ├── dialogue.txt
|   └── final_code.xxx
├── strategy_S5/
|   ├── dialogue.txt
|   └── final_code.xxx
├── changelog.csv
└── reflection.json
```

文件 1：`metadata.json`

示例：

```
{
  "student_id": "你的学号",
  "programming_language": "Python",
  "ai_tool_S1": "ChatGPT-4o",
  "ai_tool_S2": "DeepSeek-R1",
  "ai_tool_S3": "Claude-3.5-Sonnet",
  "ai_tool_S4": "ChatGPT-4o",
  "ai_tool_S5": "ChatGPT-4o"
}
```

- `programming_language`：填写 `Python` / `Java` / `C++` 三选一。
- `ai_tool_S1` 至 `ai_tool_S5`：分别填写你在五个策略中使用的 AI 工具和具体模型版本。

文件 2：`dialogue.txt`（每个策略各一个，共 5 个）

示例：

```
============================================================
策略：S1
AI 工具：ChatGPT-4o
日期：2026-09-16
============================================================
```

【我的提问】
（完整复制你输入的提示词，一字不差）

==========

【AI 的完整回答】
（完整复制 AI 的全部输出，包括所有代码和文字，不得做任何修改）

==========

> ⚠️ **重要**：
>
> - 必须**完整**记录从你输入提示词到 AI 完成回答的**全部内容**
> - **不得编辑、删改或美化** AI 的回答
> - 如果 AI 分多段输出，全部合并记录在此

文件 3：`final_code.xxx`（每个策略各一个，共 5 个）

- 文件扩展名根据你的编程语言确定：`.py` / `.java` / `.cpp`。
- 必须是**完整可运行**的程序（包含函数定义 + 测试代码）。
- 代码开头必须包含以下注释块：

```
# ============================================================
# 学生ID: [你的学号]
# 策略: S1 / S2 / S3 / S4 / S5
# 修改说明: 见 changelog.csv
# ============================================================
```

- 对于**S4**：`final_code` 应使用 AI 给出的**改进版代码**（第三步的输出），而非初始代码。
- 对于**S5**：`final_code` 应使用 AI 给出的**教学版代码**（第三步的输出），而非标准代码。

文件 4：`changelog.csv`

只记录你有修改的策略。若某策略完全没有修改，则不出现该策略的记录。

示例：

```
strategy,function_or_section,original_code,modified_code,reason
S1,closest_pair函数,"if len(points) <= 3:","if len(points) <= 3: return brute_force(points)","修复了边界条件：点数<=3时应直接调用蛮力法"
S2,strip处理循环,"for j in range(i+1, min(i+7, len(strip))):","for j in range(i+1, min(i+8, len(strip))):","修正索引范围：应检查后续7个点，即i+1到i+7共7个"
S4,距离计算,"math.sqrt(dx*dx + dy*dy)","dx*dx + dy*dy","优化：避免开平方，直接比较距离平方"
```

字段说明：

字段	含义	示例
`strategy`	该修改属于哪个策略	`S1` / `S2` / `S3` / `S4` / `S5`
`function_or_section`	修改所在的函数名或代码区域	`closest_pair函数` / `strip处理循环`
`original_code`	AI 原始代码片段（关键部分即可）	`for j in range(i+1, min(i+7, len(strip)))`
`modified_code`	修改后的代码片段（关键部分即可）	`for j in range(i+1, min(i+8, len(strip)))`
`reason`	为什么修改（一句话说明）	`修正索引范围：应检查后续7个点`

> **重要**：S4 的 `final_code` 是 AI 给出的"改进版代码"。如果你在此基础上**进一步修改**，则在 `changelog.csv` 中记录 `strategy` 为 `S4`。如果你对 S4 的改进版代码没有进一步修改，则不出现 S4 的记录。

文件 5：`reflection.json`

填写说明

请根据你完成本次作业的真实体验，回答以下问题。所有分值均为**主观感受**，没有对错，如实填写即可。

示例：

```
{
  "student_id": "240511101",
  "responses": {
    "most_helpful_strategy": "S3",
```

```
    "most_helpful_reason": "S3把算法思路、伪代码、代码、复杂度分析分开写，特别是第五部分解释了为什么只需要检查7个点，让我理解了分治算法的关键证明。",
    "least_helpful_strategy": "S1",
    "least_helpful_reason": "S1只给了代码，没有解释分治的三步骤和7点证明，我虽然能跑通但不理解算法为什么正确。",
    "code_copy_percentage": "61-80%",
    "found_code_error": true,
    "code_error_description": "AI在S1的代码中strip处理时只检查了后续6个点（索引i+1到i+6），应该是7个点，我修改为i+1到i+7。",
    "found_concept_error": false,
    "concept_error_description": "无",
    "effort_S1": 1,
    "effort_S2": 3,
    "effort_S3": 4,
    "effort_S4": 5,
    "effort_S5": 2,
    "effort_reason": "S1最简单；S4的批判分析最难，因为要先理解代码的每个细节才能看懂AI的批判。",
    "helpfulness_S1": 2,
    "helpfulness_S2": 4,
    "helpfulness_S3": 5,
    "helpfulness_S4": 3,
    "helpfulness_S5": 4,
    "helpfulness_reason": "S3对分治三步和7点证明的解释最清晰，帮助最大；S1没有解释，帮助最小。",
    "self_designed_prompt": "请先用一个具体的例子（如6个点）逐步演示分治算法的完整执行过程，画出分割线和带状区域，再给出代码实现。"
  }
}
```

逐项填写指南

字段名	含义	填写格式
`student_id`	你的学号	直接填写
`most_helpful_strategy`	你认为最有帮助的策略	`S1` / `S2` / `S3` / `S4` / `S5`
`most_helpful_reason`	为什么该策略最有帮助	不少于 15 个字的文字说明
`least_helpful_strategy`	你认为最没帮助的策略	`S1` / `S2` / `S3` / `S4` / `S5`

字段名	含义	填写格式
`least_helpful_reason`	为什么该策略最没帮助	不少于 15 个字的文字说明
`code_copy_percentage`	最终代码中直接来自 AI（未修改）的比例	`"0-20%"` / `"21-40%"` / `"41-60%"` / `"61-80%"` / `"81-100%"`
`found_code_error`	是否发现 AI 代码有语法或逻辑错误?	`true` / `false`
`code_error_description`	发现了什么代码错误?	有则描述，无则填 `"无"`
`found_concept_error`	是否发现 AI 对算法原理的解释存在概念偏差或不准确之处？（例如：对“7个点”证明的表述有误、对分治三步骤的划分不清晰、对递推式的推导有误等）	`true` / `false`
`concept_error_description`	发现了什么概念偏差?	有则描述，无则填 `"无"`
`effort_S1` 至 `effort_S5`	阅读该策略 AI 回答花费的精力	1-5 分（1=很少，5=很多）
`effort_reason`	为什么不同策略的精力花费有差异?	不少于 15 个字的文字说明
`helpfulness_S1` 至 `helpfulness_S5`	该策略对理解算法的帮助	1-5 分（1=很小，5=很大）
`helpfulness_reason`	为什么不同策略的帮助程度有差异?	不少于 15 个字的文字说明
`self_designed_prompt`	如果你来设计一种新的提示策略，完整提示词是什么?	不少于 20 个字的完整提示词

六、测试用例

你的代码将使用以下 3 组测试用例进行验证：

测试用例 1（基础）：

输入：

```
6
0 0
1 1
2 2
3 3
4 4
5 5
```

测试用例 2（中等随机）：

输入：

```
10
4 7
2 1
8 3
6 9
1 4
9 8
3 5
7 2
5 6
0 0
```

测试用例 3（边界：n=2）：

输入：

```
2
0 0
3 4
```

七、学术诚信声明

提交本作业即表示你确认并同意：

> **“以上所有对话记录是真实、完整的，未经过任何编辑或删改。最终代码是在理解 AI 回答的基础上完成的，修改日志如实记录了我的每一次修改。每个策略下我只输入了一次提示词，没有进行额外追问。reflection.json完全基于实际真实体验”**

捏造对话记录、篡改 AI 回答、直接复制 AI 代码而未加理解等学术不端行为，本作业 **记 0 分**。

八、常见问题

Q1：S4 和 S5 的三步是分三次问 AI，还是一次性问完?

一次问完。你一次性输入完整的提示词（包含第一步、第二步、第三步的全部内容），AI 会按三步结构给出完整回答。

Q2：S4 的"改进版代码"和 S5 的"教学版代码"需要出现在 `changelog.csv` 中吗?

如果你在 AI 给出的改进版/教学版代码基础上**进一步修改**了，则记录在 `changelog.csv` 中；如果没有进一步修改，则不出现即可。

Q3：`changelog.csv` 中至少需要 3 处修改，是每个策略 3 处还是所有策略加起来 3 处?

所有策略加起来至少 3 处。

Q4：如果 AI 拒绝回答或报错怎么办?

如实记录。可以换一个 AI 工具重新执行该策略，但**必须保留第一次的对话记录**并同样提交。

Q5：五种策略必须用同一个 AI 工具吗?

不必须。但须在 `metadata.json` 中分别注明每个策略使用的工具和模型版本。

Q6：关于"只需检查 7 个点"——AI 给出的代码中可能检查的是 6 个、7 个还是 8 个?

这是一个常见的实现差异。理论上证明的是"最多 7 个"，实际实现中检查 7 个（索引差 ≤ 7）或 8 个（索引差 < 8）都可以接受，关键是实现能保证正确性。如果你发现 AI 的实现与此不符，可以在 `changelog.csv` 中记录修改。这也是本作业的一个重要考察点——你是否能识别 AI 在"7 个点"这个关键细节上的实现是否正确。

Q7：提交的代码是只写函数还是写完整程序?

提交**完整的可运行程序**，包含函数定义和测试代码。测试代码中应包含上述 3 个测试用例。

Q8：如果 AI 服务网络错误或超时怎么办?

等待后重试。如果持续失败，更换 AI 工具重新执行该策略。在 `dialogue.txt` 中注明"首次尝试 [工具名] 因网络错误失败，改用 [新工具名] 完成"。

Q9：坐标必须是整数吗?

是的，题目规定输入坐标均为**整数**。这样设计的目的是避免浮点数输入带来的精度问题，使测试更加明确。

Q10：输出距离时如何保留 6 位小数?

使用编程语言的标准格式化功能：Python 用 `f"{d:.6f}"`，Java 用 `System.out.printf("%.6f", d)`，C++ 用 `cout << fixed << setprecision(6) << d`。